

Computer Graphics I

4003-570 / 4005-761

Assignment #2: Polygon Fill

Date posted: March 21st 2013

Due date: March 28th 2013

Submit to the appropriate dropbox on myCourses.

Purpose:

In this assignment, you will implement the Scanline Polygon Fill Algorithm presented in class.

Goals:

- Improve your understanding of the Scanline Polygon Fill Algorithm
- Start building up 2D drawing algorithms that can be included as part of your midterm.

2D Programming Environment:

The programming environment that you will use for this assignment, like the line drawing assignment, is a set of simple object-oriented classes with implementations in both Java and C++. You are free to use either of the implementations. The classes include:

- `simpleCanvas` – a simple 2D canvas that allows the ability to set a pixel.
- `Rasterizer` – a simple class that implements rasterization algorithms
- `fillTest` – the main function for the application.

You can find the files in the content area of myCourses. Look under the resources category. Note that the distribution contains folders for both C++ and Java versions of the programming environment.

The only class that you will need to modify is the `Rasterizer` class. For this assignment, you will need to complete the method `drawPolygon()`, implementing the Scanline Polygon Fill Algorithm. In your implementation, you need only make use of the method `setPixel()` on the passed in `simpleCanvas` to manipulate the pixels on the canvas. The signature for the `drawPolygon` method in Java is:

```
void drawPolygon(int n, int x[], int y[], simpleCanvas C)
```

and in C++:

```
void drawPolygon(int n, int x[], int y[], simpleCanvas &C)
```

Where `x` and `y` are arrays holding the integer coordinates of the vertices of the polygon to be drawn. `n` is the number of vertices in the polygon and, as such, the size of the `x` and `y` arrays. You are free to add additional members and methods to the `Rasterizer` class as you see fit, however, you cannot modify the `simpleCanvas` or `fillTest` classes.

Additional Libraries:

The Java implementation of the programming environment makes use of the standard `awt` and `Java2D` classes to access the 2D drawing areas, so no additional libraries or Java files, other than the ones provided by the assignment are required.

However, since C++ does not have a standard 2D graphics environment, you will need some additional libraries if you choose to use the C++ implementation for your submission. The C++ implementation is built on the SFML (Simple and Fast Multimedia Library), a free, cross-platform and lightweight library for performing common multimedia tasks. It can be downloaded at <http://www.sfml-dev.org/download.php>. Version 1.6 of the libraries are used. There are precompiled binaries for Windows, Mac, and Linux. (NOTE: For Windows, there are precompiled binaries for Visual Studio 2005, and Visual Studio 2008. If you are using Visual Studio 2010, a precompiled version of the libraries is available in the content area of myCourses. Look under the resources category.

Please be sure to refer to the installation instructions for your given platform:

- Windows (Visual Studio) - <http://www.sfml-dev.org/tutorials/1.6/start-vc.php>
- Mac OS X -- <http://www.sfml-dev.org/tutorials/1.6/start-osx.php>
- Linux -- <http://www.sfml-dev.org/tutorials/1.6/start-linux.php>

Submission:

Your submission should only include the files for the `Rasterizer` class that you have modified. For the Java version, this would mean the file `Rasterizer.java` and for the C++ version, this means the files `Rasterizer.cpp` and `Rasterizer.h`. PLEASE DO NOT SUBMIT ANY OTHER FILES. Specifically, since you should not be changing the `simpleCanvas` or `fillTest` classes, the source for these classes should not be submitted.

Submissions should be made to the dropbox labeled Assignment 2 - Polygon Fill

Grading:

This assignment is worth 10 points distributed as follows:

- 5.2 points for submitting code that compiles, runs, and produces a window.
- 0.3 points for each polygon in the test program. There are 16 such polygons.

Note that the correct output for the test program should look like this:

